COURS DÉBUTANT · DE ZÉRO AU FULLSTACK

Apprendre à coder

Comprendre et écrire le code toi-même : JavaScript, HTML, CSS, React, un serveur, une base de données, puis la mise en ligne.

Pour : un débutant qui apprend vite et veut construire de vraies apps.
Comment : une leçon à la fois, avec des exercices. Les corrections sont à la fin.
Ensuite : le PDF « Formation Fullstack Pro » prend le relais.

Sommaire

Partie 1 · Les bases du code (JavaScript)
1. Lancer du code · 2. Variables et types · 3. Opérations · 4. Conditions · 5. Fonctions · 6. Tableaux · 7. Boucles · 8. Objets · 9. Transformer des listes · 10. Lire une erreur

Partie 2 · Le site web (HTML, CSS, DOM)
11. HTML · 12. CSS · 13. Mise en page flexbox · 14. Le DOM et les clics · Projet 1 : todo-list

Partie 3 · Les outils du développeur
15. Le terminal, Node et npm · 16. Asynchrone et API · 17. TypeScript · 18. Git et GitHub

Partie 4 · React (le front-end moderne)
19. Composants et props · 20. État et événements · 21. Listes et formulaires

Partie 5 · Le back-end et la base de données
22. C'est quoi un serveur · 23. Ta première API · 24. SQL · 25. Connecter l'API à la base · 26. Sécurité de base

Partie 6 · Projet final et mise en ligne
Projet 2 : mini-lolpanel fullstack · 27. Mettre en ligne · La suite

Corrections des exercices

Comment utiliser ce cours

1. **Lis une leçon, puis fais ses exercices.** Écris le code toi-même, à la main. Ne copie-colle pas.
2. **N'utilise pas l'IA pour écrire le code.** Tu peux lui demander d'*expliquer* quelque chose, mais c'est toi qui tapes.
3. **Regarde la correction seulement après avoir vraiment essayé** (au moins 10 minutes).

4. **Teste tout.** Le code, ça s'apprend en le faisant tourner, en le cassant et en le réparant.
5. Une leçon par jour, c'est très bien. Deux, si tu es en forme.

> **Sur téléphone :** tu peux tester le JavaScript, le HTML et le CSS dans un éditeur en ligne comme **CodePen** (codepen.io) ou **JSFiddle** (jsfiddle.net). À partir de la Partie 3, il te faudra l'ordinateur.

PARTIE 1

Les bases du code

Le langage : JavaScript. C'est le seul langage qui tourne dans tous les navigateurs, et il marche aussi côté serveur.

LEÇON 1

Lancer du code

Un programme, c'est une liste d'instructions que l'ordinateur exécute **dans l'ordre, de haut en bas**, une ligne après l'autre.

Sur ordinateur : ouvre Chrome, appuie sur `F12` (ou `Ctrl + Maj + J`, ou `Cmd + Option + J` sur Mac), puis clique sur l'onglet **Console**. Tape :

```
console.log("Bonjour !");
```

Appuie sur Entrée : `Bonjour !` s'affiche. `console.log(...)` sert à **afficher** quelque chose. C'est ton meilleur outil pour comprendre ce que fait ton code.

Sur téléphone : dans CodePen ou JSFiddle, écris dans la zone JS et ouvre le panneau « Console ».

Les **commentaires** sont des notes pour les humains. L'ordinateur les ignore :

```
// Ceci est un commentaire sur une ligne
console.log(2 + 3); // affiche 5
```

✍️ **À toi**

Affiche ton pseudo, puis le résultat de 7 × 6 (en code, la multiplication s'écrit `*`).

LEÇON 2

Variables et types

Une **variable** est une boîte avec un nom, qui contient une valeur.

```
const pseudo = "Zelko";  // ne changera pas
let score = 0;           // va changer
score = score + 10;      // score vaut maintenant 10
```

- `const` : la valeur ne peut pas changer. **À utiliser par défaut.**
- `let` : la valeur peut changer.
- `var` : ancienne méthode. Ne l'utilise jamais.
- `=` veut dire « range dans », pas « est égal à ».

Type	Exemple	C'est quoi
string	`"Bonjour"`	Du texte, entre guillemets
number	`42` , `3.5`	Un nombre (virgule = point)
boolean	`true` , `false`	Vrai ou faux
undefined	`undefined`	Pas encore de valeur
null	`null`	Vide, exprès
array	`[1, 2, 3]`	Une liste (leçon 6)
object	`{ nom: "Léo" }`	Une fiche (leçon 8)

Les noms : en camelCase (`nombreDeVies`), sans accent ni espace, sans chiffre au début, et clairs. Pour un boolean, commence par `is` ou `has` : `isDead` , `hasKey` .

`typeof` te dit le type d'une valeur : `typeof 42` donne `"number"` .

✍️ À toi

Crée les variables pour un joueur : son pseudo, son niveau (5), son argent (120.5) et s'il est connecté. Choisis `const` ou `let` pour chacune, puis affiche-les.

LEÇON 3

Opérations

Les calculs

Opérateur	Exemple	Résultat
`+ - * /`	`10 / 4`	`2.5`
`%` (reste)	`10 % 3`	`1`
`**` (puissance)	`2 ** 3`	`8`

Raccourcis : `score += 5` veut dire `score = score + 5`. `vies -= 1` enlève 1. `compteur++` ajoute 1.

Le texte

Le `+` **colle** deux textes. Mais le mieux, c'est le **template literal** : des backticks `` ` `` et `${ }` pour insérer une valeur :

```
const pseudo = "Zelko";
const niveau = 5;
console.log(`${pseudo} est niveau ${niveau}`);
// Zelko est niveau 5
```

Piège : `"5" + 3` donne `"53"`, du texte collé. Le texte tapé dans un formulaire est toujours une string : convertis-le avec `Number("5")` avant de calculer.

Les comparaisons (elles donnent true ou false)

Opérateur	Veut dire	Exemple
`===`	est égal à	`3 === 3` → true
`!==`	est différent de	`3 !== 4` → true
`> < >= <=`	plus grand, plus petit…	`5 >= 5` → true

Utilise toujours `===` (trois signes), jamais `==` : le double égal fait des conversions bizarres (`"5" == 5` donne true).

Combiner des conditions

- `&&` (ET) : les deux doivent être vraies.
- `||` (OU) : au moins une doit être vraie.
- `!` (NON) : inverse. `!true` donne false.

```
const age = 14;
const hasTicket = true;
console.log(age >= 12 && hasTicket); // true
```

✍️ À toi

1. Combien vaut `17 % 5` ? Vérifie dans la console.
2. Avec `prix = 30` et `quantite = 3`, affiche « Total : 90 € » avec un template literal.
3. Que donne `!(5 > 3) || 2 === 2` ? Devine d'abord, puis vérifie.

LEÇON 4

Conditions

Une condition permet au programme de **choisir** quoi faire.

```
let vies = 0;

if (vies <= 0) {
  console.log("Game over");
} else if (vies === 1) {
  console.log("Attention, dernière vie !");
} else {
  console.log("Continue !");
}
```

- Le code entre `{ }` s'exécute seulement si la condition entre `( )` est vraie.
- `else if` teste une autre condition. `else` attrape tout le reste.
- Le programme s'arrête au **premier** cas vrai.

Le ternaire, un if/else court qui produit une valeur :

```
const message = vies > 0 ? "En vie" : "Mort";
```

✍️ À toi

Écris un programme qui donne une mention à partir d'une variable `note` (sur 20) : moins de 10 → « Raté », de 10 à 13 → « Bien », de 14 à 16 → « Très bien », 17 et plus → « Excellent ». Teste avec 8, 13, 14 et 20.

LEÇON 5

Fonctions

Une fonction est une **recette réutilisable** : tu l'écris une fois et tu l'utilises autant que tu veux. Elle reçoit des **paramètres** (les ingrédients) et peut **renvoyer** un résultat avec `return`.

```
function calculerTotal(prix, quantite) {
  return prix * quantite;
}

const total = calculerTotal(30, 3); // on « appelle » la fonction
console.log(total); // 90
```

La même chose écrite en **fonction fléchée** (arrow function), très utilisée en React :

```
const calculerTotal = (prix, quantite) => {
  return prix * quantite;
};

// Version courte, si la fonction ne fait que renvoyer une valeur :
const doubler = (n) => n * 2;
```

- `return` renvoie la valeur **et arrête** la fonction. Le code placé après ne s'exécute pas.
- Sans `return`, la fonction renvoie `undefined`.
- Une variable créée **dans** une fonction n'existe que dans cette fonction. C'est ce qu'on appelle la *portée* (scope).
- Une bonne fonction fait **une seule chose**, et son nom commence par un verbe : `calculerTotal`, `afficherScore`, `isValidEmail`.

Paramètre avec une valeur par défaut :

```
function saluer(pseudo = "inconnu") {
  return `Salut ${pseudo} !`;
}
saluer();        // "Salut inconnu !"
saluer("Zelko"); // "Salut Zelko !"
```

✍️ À toi

1. Écris `estMajeur(age)` qui renvoie `true` si l'âge est d'au moins 18 ans, sinon `false`.
2. Écris `mention(note)` qui reprend l'exercice de la leçon 4 et **renvoie** la mention au lieu de l'afficher.
3. Écris `appliquerReduction(prix, pourcentage)` : `appliquerReduction(50, 20)` doit renvoyer 40.

LEÇON 6

Tableaux (arrays)

Un tableau est une **liste ordonnée** de valeurs.

```
const inventaire = ["épée", "bouclier", "potion"];

console.log(inventaire[0]);       // "épée"  ← on compte à partir de
0 !
console.log(inventaire.length);   // 3
console.log(inventaire[inventaire.length - 1]); // "potion" (le
dernier)

inventaire.push("clé");    // ajoute à la fin
inventaire.pop();          // retire le dernier
inventaire.includes("épée"); // true
inventaire.indexOf("potion"); // 2 (ou -1 si absent)
```

Pourquoi `push` marche sur un `const` ? Parce que `const` interdit de remplacer la boîte, pas de modifier ce qu'il y a dedans. `inventaire.push("clé")` est autorisé, `inventaire = []` ne l'est pas.

✍️ À toi

Crée un tableau de 4 jeux que tu aimes. Affiche le 2e, ajoute-en un, puis affiche combien il y en a. Que donne `jeux[10]` ? Pourquoi ?

LEÇON 7

Boucles

Une boucle **répète** du code. C'est indispensable pour traiter une liste.

for...of : parcourir une liste (la plus utile)

```
const inventaire = ["épée", "bouclier", "potion"];
for (const objet of inventaire) {
  console.log(`Tu as : ${objet}`);
}
```

for classique : répéter N fois

```
for (let i = 1; i <= 5; i++) {
  console.log(`Tour ${i}`);
}
// départ ; condition pour continuer ; ce qu'on fait après chaque
tour
```

while : tant que la condition est vraie

```
let vies = 3;
while (vies > 0) {
  console.log(`Il reste ${vies} vies`);
  vies--;
}
```

> **Boucle infinie :** si la condition ne devient jamais fausse (par exemple si tu oublies `vies--`), le programme tourne pour toujours et la page plante. Dans ce cas, ferme l'onglet.

`break` sort de la boucle tout de suite. `continue` passe directement au tour suivant.

✍️ **À toi**

1. Affiche les nombres de 10 à 1, puis « Décollage ! ».
2. Avec `const scores = [12, 45, 7, 30]`, calcule la somme avec une boucle (attendu : 94).
3. Trouve le plus grand score avec une boucle (attendu : 45).

LEÇON 8

Objets

Un objet est une **fiche** : des paires *clé : valeur* qui décrivent une chose.

```
const joueur = {
  pseudo: "Zelko",
  niveau: 5,
  isOnline: true,
  inventaire: ["épée", "potion"],
};

console.log(joueur.pseudo);         // "Zelko"
console.log(joueur["niveau"]);      // 5 (autre façon d'écrire)
joueur.niveau = 6;                  // modifier
joueur.argent = 100;                // ajouter une clé
console.log(joueur.inventaire[0]); // "épée"
```

Une liste d'objets, c'est la forme de données la plus courante dans une vraie app :

```
const projets = [
  { id: 1, nom: "Landing page", pages: 3 },
  { id: 2, nom: "Dashboard", pages: 8 },
];
for (const projet of projets) {
  console.log(`${projet.nom} : ${projet.pages} pages`);
}
```

Déstructuration, un raccourci pour sortir des valeurs d'un objet (très utilisé en React) :

```
const { pseudo, niveau } = joueur;
console.log(pseudo); // "Zelko"
```

JSON : c'est le format texte utilisé pour échanger des données entre le navigateur et le serveur. Il ressemble beaucoup à un objet JavaScript. `JSON.stringify(obj)` transforme un objet en texte, et `JSON.parse(texte)` fait l'inverse.

✍️ À toi

Crée un objet `bouton` avec un texte, une couleur, une largeur et un boolean `isVisible`. Écris une fonction `decrireBouton(bouton)` qui renvoie par exemple : « Bouton "Envoyer" bleu de 120 px ».

LEÇON 9

Transformer des listes

Ces méthodes remplacent la plupart des boucles. Tu les verras **partout** dans le code React généré par l'IA.

```
const prix = [10, 25, 40];

// map : transforme chaque élément → nouveau tableau de même taille
const prixTTC = prix.map((p) => p * 1.2);   // [12, 30, 48]

// filter : garde seulement ceux qui passent le test
const chers = prix.filter((p) => p > 20);   // [25, 40]

// find : renvoie le PREMIER qui passe le test (ou undefined)
const premierCher = prix.find((p) => p > 20); // 25

// some / every : au moins un ? / tous ?
prix.some((p) => p > 30);   // true
prix.every((p) => p > 5);   // true

// reduce : réduit la liste à une seule valeur
const total = prix.reduce((somme, p) => somme + p, 0); // 75
```

Avec une liste d'objets :

```
const projets = [
  { id: 1, nom: "Landing", isPublic: true },
  { id: 2, nom: "Admin", isPublic: false },
];
const publics = projets.filter((p) => p.isPublic);
const noms = projets.map((p) => p.nom);       // ["Landing",
"Admin"]
const projet2 = projets.find((p) => p.id === 2);
```

On peut les **enchaîner** : `projets.filter(p => p.isPublic).map(p => p.nom)`.

✍️ **À toi**

Avec `const joueurs = [{ nom: "Léa", score: 80 }, { nom: "Tom", score: 45 }, { nom: "Sam", score: 92 }]` :

1. Liste des noms. 2. Joueurs avec plus de 50 points. 3. Le joueur nommé « Tom ». 4. Le score total.

LEÇON 10

Lire une erreur et déboguer

Une erreur n'est pas un échec. C'est un **message qui t'explique** ce qui ne va pas. Lis toujours le message en entier.

Erreur	Veut dire	Cause fréquente
`SyntaxError`	Le code est mal écrit	Parenthèse, accolade ou guillemet oublié
`ReferenceError: x is not defined`	La variable n'existe pas	Faute de frappe, majuscule, mauvais endroit
`TypeError: Cannot read properties of undefined`	Tu lis un champ sur quelque chose de vide	`find` n'a rien trouvé, ou les données ne sont pas encore chargées
`TypeError: x is not a function`	Tu appelles quelque chose qui n'est pas une fonction	Faute de frappe dans le nom
`Assignment to constant variable`	Tu modifies un `const`	Il fallait `let`

La méthode pour déboguer :

1. Lis le message **et le numéro de ligne**.
2. Mets des `console.log` avant la ligne qui plante pour voir ce que contiennent vraiment tes variables.

3. Compare ce que tu *crois* qu'il y a avec ce qu'il y a *vraiment*. Le bug est dans cet écart.
4. Change **une seule chose** à la fois, puis reteste.

✍️ À toi

Trouve les 3 bugs :

```
const joueur = { nom: "Léa", score: 10 };
function ajouterPoints(points) {
  joueur.score = joueur.score + Points;
}
const bonus = "5";
ajouterPoints(bonus);
console.log(joueur.Score);
```

PARTIE 2

Le site web

HTML = la structure. CSS = l'apparence. JavaScript = le comportement.

LEÇON 11

HTML : la structure

Le HTML décrit **ce qu'il y a** sur la page, avec des **balises** : `<nom>contenu</nom>`.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <title>Mon site</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Titre principal</h1>
    <p>Un paragraphe avec un <a href="https://exemple.com">lien</a>.
</p>
    <img src="logo.png" alt="Logo du site">
    <button id="monBouton" class="btn">Clique</button>
    <script src="app.js"></script>
  </body>
</html>
```

Balise	Rôle
`h1` à `h6`	Titres, du plus important au moins important
`p`, `a`, `img`	Paragraphe, lien, image (avec `alt` pour l'accessibilité)
`ul` / `ol` + `li`	Listes
`div`, `span`	Boîtes génériques (bloc, en ligne)
`header`, `nav`, `main`, `section`, `footer`	Zones de la page qui ont un sens (HTML « sémantique »)
`form`, `input`, `label`, `button`	Formulaires

Attributs : `id` (unique sur la page), `class` (pour le style, réutilisable), `href`, `src`, `type`…

> ✍️ **À toi**
>
> Fais une page « profil » : ton pseudo en titre, une courte bio, une liste de 3 compétences et un bouton « Me contacter ».

LEÇON 12

CSS : l'apparence

```
.btn {                    /* sélecteur : tout ce qui a class="btn"
*/
  background: #4f46e5;    /* propriété: valeur; */
  color: white;
  padding: 12px 20px;     /* espace intérieur */
  border-radius: 8px;     /* coins arrondis */
  border: none;
  font-size: 16px;
}
.btn:hover { background: #4338ca; }  /* au survol */
#monBouton { margin-top: 16px; }     /* par id */
p { line-height: 1.5; }              /* toutes les balises p */
```

Le modèle de boîte : chaque élément est une boîte. De l'intérieur vers l'extérieur : *contenu* → `padding` (espace intérieur) → `border` (bordure) → `margin` (espace extérieur). Mets `* { box-sizing: border-box; }` en haut de ton CSS : la largeur inclut alors le padding et la bordure, ce qui est beaucoup plus simple.

Unités : `px` (fixe), `%` (du parent), `rem` (relative à la taille du texte), `vw` / `vh` (relatives à l'écran).

✍️ À toi

Donne du style à ta page profil : une police, un fond clair, une carte centrée avec des coins arrondis et une ombre (`box-shadow`), et un bouton coloré avec un effet au survol.

LEÇON 13

Mise en page avec flexbox

Flexbox range des éléments **en ligne ou en colonne**. Il suffit de le mettre sur le **parent** :

```
.barre {
  display: flex;
  flex-direction: row;            /* ou column */
  justify-content: space-between; /* sur l'axe principal */
  align-items: center;            /* sur l'autre axe */
  gap: 12px;                      /* espace entre les enfants */
}
.centre {                          /* centrer n'importe quoi */
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Responsive (s'adapte au téléphone) avec une *media query* :

```
.cartes { display: flex; gap: 16px; }
@media (max-width: 600px) {
  .cartes { flex-direction: column; } /* empilées sur petit écran */
}
```

Pour les grilles à deux dimensions, il existe aussi `display: grid`. Tu le découvriras après flexbox.

✍️ À toi

Fais une barre de navigation : le logo à gauche, 3 liens à droite, le tout centré verticalement. Sur téléphone, les liens passent sous le logo.

LEÇON 14

Le DOM : JavaScript qui modifie la page

Le **DOM** est la version de ta page HTML que JavaScript peut lire et modifier.

```
// 1. Trouver un élément
const bouton = document.querySelector("#monBouton"); // par id
const titre = document.querySelector("h1");          // par balise
const cartes = document.querySelectorAll(".carte");  // tous

// 2. Le modifier
titre.textContent = "Nouveau titre";
titre.classList.add("rouge");
titre.style.color = "red";

// 3. Réagir à un événement
let compteur = 0;
bouton.addEventListener("click", () => {
  compteur++;
  titre.textContent = `Cliqué ${compteur} fois`;
});

// 4. Créer un élément
const li = document.createElement("li");
li.textContent = "Nouvel élément";
document.querySelector("ul").append(li);
```

Les événements courants : `click`, `input` (chaque frappe), `submit` (envoi de formulaire), `keydown`.

> **Sécurité :** utilise `textContent` pour afficher un texte tapé par un utilisateur, **jamais** `innerHTML`. Avec `innerHTML`, si quelqu'un tape du code HTML ou du `<script>`, il sera exécuté. C'est la faille **XSS**.

✍️ À toi

Fais un compteur : un nombre affiché et trois boutons **+1**, **−1** et **Reset**. Bonus : le nombre devient rouge quand il est négatif.

🏗️ Projet 1 : la todo-list

Ton premier vrai projet. Fais-le seul, puis compare avec ce code.

Ce qu'elle doit faire : ajouter une tâche, la cocher comme faite, la supprimer, afficher le nombre de tâches restantes. Bonus : les tâches restent après un rechargement de la page (`localStorage`).

Idée clé : les données sont dans un **tableau d'objets**, et une fonction `afficher()` redessine la liste à partir de ce tableau. Tu modifies les données, puis tu réaffiches. C'est exactement l'idée de React.

```
<form id="form">
  <input id="champ" placeholder="Nouvelle tâche" required>
  <button>Ajouter</button>
</form>
<p id="restant"></p>
<ul id="liste"></ul>
```

```
let taches = JSON.parse(localStorage.getItem("taches") ?? "[]");

const form = document.querySelector("#form");
const champ = document.querySelector("#champ");
const liste = document.querySelector("#liste");
const restant = document.querySelector("#restant");

function sauvegarder() {
  localStorage.setItem("taches", JSON.stringify(taches));
}

function afficher() {
  liste.innerHTML = "";              // on vide (aucun texte
utilisateur ici)
  for (const tache of taches) {
    const li = document.createElement("li");

    const caseACocher = document.createElement("input");
    caseACocher.type = "checkbox";
    caseACocher.checked = tache.faite;
    caseACocher.addEventListener("change", () => {
      tache.faite = !tache.faite;
      sauvegarder(); afficher();
    });

    const texte = document.createElement("span");
    texte.textContent = tache.texte;  // textContent : sûr
    if (tache.faite) texte.style.textDecoration = "line-through";

    const supprimer = document.createElement("button");
    supprimer.textContent = "×";
    supprimer.addEventListener("click", () => {
      taches = taches.filter((t) => t.id !== tache.id);
      sauvegarder(); afficher();
    });

    li.append(caseACocher, texte, supprimer);
    liste.append(li);
  }
  const nb = taches.filter((t) => !t.faite).length;
  restant.textContent = `${nb} tâche(s) restante(s)`;
}
```

```
form.addEventListener("submit", (e) => {
  e.preventDefault();                // empêche le rechargement de
la page
  const texte = champ.value.trim();
  if (texte === "") return;
  taches.push({ id: Date.now(), texte, faite: false });
  champ.value = "";
  sauvegarder(); afficher();
});

afficher();
```

`??` veut dire « si c'est null ou undefined, prends la valeur de droite ».
`e.preventDefault()` empêche le formulaire de recharger la page.

✍️ À toi

Améliorations : 1. Un bouton « Tout effacer ». 2. Des filtres « Toutes / À faire / Faites ». 3. Modifier le texte d'une tâche avec un double-clic.

PARTIE 3

Les outils du développeur

À partir d'ici, il te faut l'ordinateur.

LEÇON 15

Le terminal, Node et npm

Le **terminal** permet de piloter l'ordinateur en tapant des commandes. Tous les outils pro s'utilisent comme ça.

Commande	Rôle
`pwd`	Où suis-je ?
`ls` (`dir` sous Windows cmd)	Lister les fichiers
`cd dossier` / `cd ..`	Entrer dans un dossier / remonter
`mkdir nom`	Créer un dossier
`code .`	Ouvrir le dossier dans VS Code

À installer : VS Code (l'éditeur de code), **Node.js** en version LTS (pour lancer du JavaScript hors du navigateur), et **Git**.

```
node --version          # vérifie l'installation
node app.js             # lance un fichier JavaScript
npm init -y             # crée package.json (la fiche du projet)
npm install nom-paquet  # ajoute une librairie (dans node_modules)
npm run dev             # lance un script défini dans package.json
```

- `package.json` : le nom du projet, ses dépendances (les librairies qu'il utilise) et ses scripts.
- `node_modules/` : le code des librairies. Énorme, on ne le partage jamais : on le met dans `.gitignore`.
- `package-lock.json` : les versions exactes installées. On le garde.

Modules : on découpe le code en plusieurs fichiers.

```
// maths.js
export function doubler(n) { return n * 2; }

// app.js
import { doubler } from "./maths.js";
console.log(doubler(4));
```

Pour utiliser `import` / `export` avec Node, ajoute `"type": "module"` dans `package.json`.

> ✍️ **À toi**
>
> Crée un dossier `mon-premier-projet`, lance `npm init -y`, écris un fichier `app.js` qui importe une fonction depuis un autre fichier, puis lance-le avec `node app.js`.

LEÇON 16

Asynchrone : fetch et les API

Certaines actions **prennent du temps** : demander des données à un serveur, lire un fichier. Le code n'attend pas sans rien faire : il lance la demande, et le résultat arrive *plus tard*. C'est l'**asynchrone**.

Une **API** est une adresse web qui renvoie des données (souvent en JSON) au lieu d'une page.

```
async function chargerUtilisateur() {
  try {
    const reponse = await
fetch("https://jsonplaceholder.typicode.com/users/1");
    if (!reponse.ok) {
      throw new Error(`Erreur HTTP ${reponse.status}`);
    }
    const utilisateur = await reponse.json();
    console.log(utilisateur.name);
  } catch (erreur) {
    console.log("Impossible de charger :", erreur.message);
  }
}
chargerUtilisateur();
```

- `async` devant une fonction permet d'utiliser `await` dedans.
- `await` veut dire « attends le résultat avant de passer à la ligne suivante ».
- `fetch` renvoie une *Promise* : une promesse de résultat futur.
- `try/catch` attrape les erreurs (réseau coupé, serveur en panne). **Gère toujours le cas d'erreur.**
- Envoyer des données : `fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(donnees) })`.

jsonplaceholder.typicode.com est une API publique gratuite, faite pour s'entraîner.

✍️ À toi

Charge la liste `https://jsonplaceholder.typicode.com/users` et affiche les noms dans une liste HTML. Affiche « Chargement… » pendant l'attente, et un message si ça échoue.

LEÇON 17

TypeScript

TypeScript, c'est du JavaScript **avec des types**. L'éditeur te prévient des erreurs *avant* que le code tourne. Tous les projets pros l'utilisent, et c'est ton meilleur allié pour vérifier le code de l'IA.

```
let score: number = 10;
score = "dix"; // ❌ erreur : "dix" n'est pas un nombre

function calculerTotal(prix: number, quantite: number): number {
  return prix * quantite;
}

// Décrire la forme d'un objet
type Projet = {
  id: number;
  nom: string;
  isPublic: boolean;
  description?: string;      // ? = optionnel
};

const p: Projet = { id: 1, nom: "Landing", isPublic: true };

// Une valeur parmi quelques choix possibles
type Format = "html" | "react" | "vue";
let format: Format = "react";
format = "angular"; // ❌ erreur : pas dans la liste

const projets: Projet[] = [p];  // tableau de Projet
```

Les fichiers TypeScript finissent par `.ts` (ou `.tsx` avec React). Le `"5" + 3` de la leçon 3 ? TypeScript t'aurait souvent prévenu.

> Le type `any` désactive toute la vérification. Si l'IA en met, demande-lui le vrai type.

✍️ À toi

Écris le type `Joueur` (pseudo, niveau, inventaire en liste de textes, guilde optionnelle), puis une fonction `monterNiveau(joueur: Joueur): Joueur` qui renvoie le joueur avec un niveau de plus.

LEÇON 18

Git et GitHub

Git enregistre l'historique de ton code. Tu peux revenir en arrière à tout moment. **GitHub** stocke cet historique en ligne et permet de travailler à plusieurs.

```
git init                        # démarre l'historique
git status                      # qu'est-ce qui a changé ?
git add .                       # prépare les changements
git commit -m "Ajoute la todo-list"   # enregistre une étape
git log --oneline               # voir l'historique
git checkout -b nouvelle-idee   # nouvelle branche pour tester
git push                        # envoie sur GitHub
```

- Un **commit** est une sauvegarde avec un message qui dit ce que tu as fait.
- Une **branche** est une version parallèle pour tester sans casser la version principale (`main`).
- `.gitignore` : la liste des fichiers à ne jamais enregistrer (`node_modules`, `.env`).
- Fais un commit à chaque petite étape qui marche.

✍️ À toi

Mets ta todo-list sur GitHub : `git init`, premier commit, crée un dépôt sur github.com, puis `git push`.

PARTIE 4

React

La librairie la plus utilisée pour construire des interfaces. lolpanel utilise très probablement React ou un outil proche.

Créer un projet (Vite est un outil qui prépare et lance le projet) :

```
npm create vite@latest mon-app -- --template react-ts
cd mon-app
npm install
npm run dev     # ouvre l'adresse affichée (souvent
http://localhost:5173)
```

LEÇON 19

Composants et props

Un **composant** est une fonction qui renvoie un morceau d'interface. On l'écrit en **JSX** : du HTML directement dans le JavaScript.

```
type CarteProps = { titre: string; pages: number };

function CarteProjet({ titre, pages }: CarteProps) {
  return (
    <div className="carte">
      <h2>{titre}</h2>
      <p>{pages} pages</p>
    </div>
  );
}

export default function App() {
  return (
    <main>
      <CarteProjet titre="Landing" pages={3} />
      <CarteProjet titre="Dashboard" pages={8} />
    </main>
  );
}
```

- Le nom d'un composant commence par une **majuscule**.
- Les **props** sont les paramètres d'un composant, transmis comme des attributs HTML.
- `{ }` dans le JSX permet d'insérer du JavaScript.
- `className` au lieu de `class`. Un composant renvoie **un seul** élément parent (sinon, entoure-les de `<>...</>`).

LEÇON 20

État et événements

L'**état** (state) est une donnée qui peut changer et qui **redessine automatiquement** l'interface quand elle change. C'est le `afficher()` de ta todo-list, mais fait automatiquement.

```
import { useState } from "react";

function Compteur() {
  const [compteur, setCompteur] = useState(0);

  return (
    <div>
      <p>Valeur : {compteur}</p>
      <button onClick={() => setCompteur(compteur + 1)}>+1</button>
      <button onClick={() => setCompteur(0)}>Reset</button>
    </div>
  );
}
```

- `useState(0)` renvoie la valeur actuelle et une fonction pour la changer.
- Ne modifie **jamais** l'état directement (`compteur = 5` ne marche pas). Passe toujours par `setCompteur` .
- Pour un tableau ou un objet, crée une **copie** : `setTaches([...taches, nouvelle])` . Le `...` (spread) copie les éléments existants.

Afficher selon une condition :

```
{isConnected ? <p>Bienvenue</p> : <button>Connexion</button>}
{erreur && <p className="erreur">{erreur}</p>}
```

LEÇON 21

Listes et formulaires

```
import { useState } from "react";

type Tache = { id: number; texte: string; faite: boolean };

export default function Todo() {
  const [taches, setTaches] = useState<Tache[]>([]);
  const [texte, setTexte] = useState("");

  function ajouter(e: React.FormEvent) {
    e.preventDefault();
    if (texte.trim() === "") return;
    setTaches([...taches, { id: Date.now(), texte: texte.trim(),
faite: false }]);
    setTexte("");
  }

  function basculer(id: number) {
    setTaches(taches.map((t) => (t.id === id ? { ...t, faite:
!t.faite } : t)));
  }

  function supprimer(id: number) {
    setTaches(taches.filter((t) => t.id !== id));
  }

  return (
    <div>
      <form onSubmit={ajouter}>
        <input value={texte} onChange={(e) =>
setTexte(e.target.value)} />
        <button>Ajouter</button>
      </form>
      <ul>
        {taches.map((t) => (
          <li key={t.id}>
            <input type="checkbox" checked={t.faite} onChange={() =>
basculer(t.id)} />
            {t.texte}
```

```
            <button onClick={() => supprimer(t.id)}>x</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

- `.map()` transforme le tableau de données en tableau d'éléments à afficher. Tu reconnais la leçon 9 ?
- `key` : un identifiant **unique et stable** pour chaque élément de liste. Il permet à React de savoir quel élément a changé. N'utilise pas l'index du tableau.
- `{ ...t, faite: !t.faite }` : copie de l'objet avec un seul champ modifié.
- Champ « contrôlé » : `value` vient de l'état, et `onChange` met l'état à jour à chaque frappe.
- React affiche le texte de façon sûre par défaut (pas de XSS). Évite `dangerouslySetInnerHTML` : son nom dit tout.

> **Comparaison :** c'est ta todo-list du Projet 1, en beaucoup plus court. Tu décris *à quoi l'interface doit ressembler* selon les données, et React se charge de mettre la page à jour.

✍️ À toi

1. Découpe la todo en composants : `FormulaireTache`, `ListeTaches`, `LigneTache`, en passant les données et les fonctions par les props.
2. Ajoute les filtres Toutes / À faire / Faites.
3. Bonus : charge des tâches au démarrage depuis jsonplaceholder (`/todos?_limit=5`) avec `useEffect` (demande à l'IA de t'expliquer `useEffect`, puis écris-le toi-même).

PARTIE 5

Back-end et base de données

Le « full » de fullstack : le serveur qui garde les données et prend les décisions.

LEÇON 22

C'est quoi un serveur

```
NAVIGATEUR  ──requête──▶  SERVEUR (API)  ──▶  BASE DE DONNÉES
(front-end) ◀─réponse──   (back-end)     ◀──  (stockage)
```

- Le **front-end** tourne chez l'utilisateur. Il affiche les choses et envoie des demandes.
- Le **back-end** tourne sur *ton* serveur. Il vérifie les demandes, applique les règles et parle à la base.
- La **base de données** garde les données pour toujours, et pour tout le monde. Le `localStorage`, lui, reste dans un seul navigateur.

Une **requête HTTP** a une **méthode** (ce qu'on veut faire) et une **adresse** (sur quoi) :

Méthode	Action	Exemple
GET	Lire	`GET /api/projets`
POST	Créer	`POST /api/projets`
PUT / PATCH	Modifier	`PATCH /api/projets/3`
DELETE	Supprimer	`DELETE /api/projets/3`

La réponse contient un **code de statut** : `200` OK, `201` créé, `400` / `422` données invalides, `401` pas connecté, `403` pas le droit, `404` introuvable, `500` erreur du serveur.

LEÇON 23

Ta première API (Express)

```
mkdir api && cd api
npm init -y
npm install express
```

Ajoute `"type": "module"` dans `package.json`, puis crée `server.js` :

```
import express from "express";

const app = express();
app.use(express.json()); // lit le JSON envoyé par le client

// Données en mémoire (perdues au redémarrage, on corrige ça en
leçon 25)
let projets = [{ id: 1, nom: "Landing" }];

app.get("/api/projets", (req, res) => {
  res.json(projets);
});

app.get("/api/projets/:id", (req, res) => {
  const projet = projets.find((p) => p.id ===
Number(req.params.id));
  if (!projet) return res.status(404).json({ erreur: "Projet
introuvable" });
  res.json(projet);
});

app.post("/api/projets", (req, res) => {
  const { nom } = req.body;
  if (typeof nom !== "string" || nom.trim() === "" || nom.length >
80) {
    return res.status(422).json({ erreur: "Nom invalide" });
  }
  const projet = { id: Date.now(), nom: nom.trim() };
  projets.push(projet);
  res.status(201).json(projet);
});

app.listen(3001, () => console.log("API sur
http://localhost:3001"));
```

Lance-la avec `node server.js`, puis ouvre `http://localhost:3001/api/projets` dans le navigateur.

- `req` (la requête) contient ce qu'envoie le client : `req.params` (l'adresse), `req.query` (après le `?`), `req.body` (les données).

- `res` (la réponse) sert à répondre : `res.json(...)`, `res.status(404)`.
- **On vérifie toujours ce qui arrive** (le nom est-il un texte ? pas vide ? pas trop long ?). Le client peut envoyer n'importe quoi.

> ✍️ **À toi**
>
> Ajoute la route `DELETE /api/projets/:id` (réponse 204 si supprimé, 404 si introuvable) et `PATCH /api/projets/:id` pour renommer.

LEÇON 24

SQL : parler à une base de données

Une base **relationnelle** range les données dans des **tables**, comme des feuilles de calcul : des colonnes (les champs) et des lignes (les éléments). On lui parle en **SQL**.

```
-- Créer une table
CREATE TABLE projets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nom TEXT NOT NULL,
  html TEXT NOT NULL DEFAULT '',
  cree_le TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Créer une ligne
INSERT INTO projets (nom) VALUES ('Landing');

-- Lire
SELECT * FROM projets;
SELECT id, nom FROM projets WHERE id = 1;
SELECT * FROM projets ORDER BY cree_le DESC LIMIT 10;

-- Modifier
UPDATE projets SET nom = 'Accueil' WHERE id = 1;

-- Supprimer
DELETE FROM projets WHERE id = 1;
```

> **Oublier le `WHERE`** dans un `UPDATE` ou un `DELETE` modifie ou supprime **toutes** les lignes de la table. Relis toujours avant d'exécuter.

Les relations : une table peut pointer vers une autre avec une **clé étrangère**. Par exemple, chaque projet appartient à un utilisateur :

```
CREATE TABLE utilisateurs (id INTEGER PRIMARY KEY, email TEXT UNIQUE
NOT NULL);
CREATE TABLE projets (
  id INTEGER PRIMARY KEY,
  nom TEXT NOT NULL,
  proprietaire_id INTEGER NOT NULL REFERENCES utilisateurs(id)
);

-- Les projets avec l'email de leur propriétaire
SELECT projets.nom, utilisateurs.email
FROM projets
JOIN utilisateurs ON utilisateurs.id = projets.proprietaire_id;
```

Pour t'entraîner dans le navigateur, cherche « SQLite online » : plusieurs sites permettent de tester du SQL sans rien installer.

✍️ À toi

Écris le SQL pour : 1. une table `joueurs` (id, pseudo unique, score, 0 par défaut). 2. Ajouter 3 joueurs. 3. Afficher les joueurs avec plus de 50 points, du meilleur au moins bon. 4. Ajouter 10 points à un joueur précis.

LEÇON 25

Connecter l'API à la base

On utilise **SQLite** : la base entière tient dans un seul fichier, ce qui est parfait pour apprendre.

```
npm install better-sqlite3
```

```
import express from "express";
import Database from "better-sqlite3";

const db = new Database("lolpanel.db");
db.exec(`CREATE TABLE IF NOT EXISTS projets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nom TEXT NOT NULL,
  html TEXT NOT NULL DEFAULT ''
)`);

const app = express();
app.use(express.json());

app.get("/api/projets", (req, res) => {
  const projets = db.prepare("SELECT id, nom FROM projets ORDER BY
id DESC").all();
  res.json(projets);
});

app.get("/api/projets/:id", (req, res) => {
  const projet = db.prepare("SELECT * FROM projets WHERE id =
?").get(Number(req.params.id));
  if (!projet) return res.status(404).json({ erreur: "Projet
introuvable" });
  res.json(projet);
});

app.post("/api/projets", (req, res) => {
  const { nom, html = "" } = req.body;
  if (typeof nom !== "string" || nom.trim() === "" || nom.length >
80 || typeof html !== "string") {
    return res.status(422).json({ erreur: "Données invalides" });
  }
  const resultat = db.prepare("INSERT INTO projets (nom, html)
VALUES (?, ?)").run(nom.trim(), html);
  res.status(201).json({ id: Number(resultat.lastInsertRowid), nom:
nom.trim() });
});

app.listen(3001, () => console.log("API sur
http://localhost:3001"));
```

Maintenant, les projets **survivent au redémarrage** : ils sont dans le fichier `lolpanel.db`.

Les `?` sont des **paramètres** : la librairie y insère les valeurs de façon sûre. C'est la règle de sécurité la plus importante avec SQL (voir la leçon suivante).

LEÇON 26

Sécurité de base (à ne jamais oublier)

1. Le client propose, le serveur décide

Tout ce qui tourne dans le navigateur peut être lu et modifié par l'utilisateur (F12). Cacher un bouton ne protège rien. Toute vérification importante (droits, paiement, licence, limites) se fait **sur le serveur**.

2. Injection SQL

```
// ❌ DANGEREUX : on colle le texte de l'utilisateur dans le SQL
db.prepare(`SELECT * FROM projets WHERE nom = '${nom}'`).all();
// Si nom vaut :  ' OR '1'='1   → la requête renvoie TOUT

// ✅ SÛR : paramètre
db.prepare("SELECT * FROM projets WHERE nom = ?").all(nom);
```

3. XSS

N'insère jamais du texte d'utilisateur avec `innerHTML` ou `dangerouslySetInnerHTML`. Utilise `textContent`, ou laisse React afficher le texte normalement.

4. Les secrets

- Les mots de passe de base de données et les clés d'API vont dans un fichier `.env` côté serveur, lu avec `process.env.NOM`.
- Le `.env` est dans le `.gitignore`. **Jamais sur GitHub.**
- Jamais de secret dans le code du front-end : tout le monde peut le lire.

5. Valider tout ce qui arrive

Le type, la longueur, le format. Plus tard, tu utiliseras la librairie **Zod** pour le faire proprement.

6. Les mots de passe

On ne les stocke **jamais** tels quels. On stocke un *hash*, avec une librairie spécialisée (argon2 ou bcrypt). Pour commencer, le plus sûr est d'utiliser un service ou une librairie d'authentification déjà prête plutôt que de tout coder soi-même.

> ✍️ **À toi**
>
> Vrai ou faux ? 1. « Mon bouton Admin est caché pour les non-admins, donc ma route admin est protégée. » 2. « Je peux mettre ma clé d'API dans le front si personne ne connaît l'adresse du site. » 3. « Avec des `?` dans mes requêtes SQL, je suis protégé contre l'injection SQL. »

PARTIE 6

Projet final et mise en ligne

Tu assembles tout : React + API + base de données.

🏗️ Projet 2 : mini-lolpanel fullstack

Objectif : une app où on crée des projets, où on construit une petite page en ajoutant des blocs (titre, texte, bouton), qui génère le code HTML et le **sauvegarde en base**.

Structure

```
mini-lolpanel/
  api/        ← Express + SQLite (leçon 25)
  web/        ← React + Vite (partie 4)
```

Les données d'un projet

```
type Bloc =
  | { id: number; type: "titre"; texte: string }
  | { id: number; type: "texte"; texte: string }
  | { id: number; type: "bouton"; texte: string; couleur: string };

type Projet = { id: number; nom: string; blocs: Bloc[] };
```

En base, stocke la liste des blocs sous forme de texte JSON (`JSON.stringify` pour enregistrer, `JSON.parse` pour relire).

Faire parler le front et l'API

Dans `web/vite.config.ts`, ajoute un *proxy* pour que `/api` soit envoyé à ton serveur :

```
export default defineConfig({
  plugins: [react()],
  server: { proxy: { "/api": "http://localhost:3001" } },
});
```

Dans React, tu peux alors écrire `fetch("/api/projets")`.

Les étapes

- ☐ API : routes `GET`, `POST`, `PATCH` et `DELETE` pour `/api/projets`, avec validation.
- ☐ Front : page « Mes projets » (liste + création).
- ☐ Front : éditeur. Boutons « + Titre », « + Texte », « + Bouton », modification du texte de chaque bloc, suppression et réordonnancement (monter/descendre).
- ☐ Aperçu en direct à côté de l'éditeur.
- ☐ Fonction `genererHTML(blocs)` qui produit le code HTML, avec un bouton « Copier le code ». Attention : **échappe** le texte (`<` devient `<`, etc.) pour éviter le XSS.
- ☐ Bouton « Sauvegarder » qui envoie un `PATCH` à l'API, avec un message « Sauvegardé ✓ » ou « Erreur ».
- ☐ Bonus : un export React (`genererReact(blocs)`), puis un export en téléchargement de fichier.

> **Règle du projet :** tu écris au moins 80 % du code toi-même. L'IA a le droit de t'expliquer une erreur ou un concept, pas d'écrire des fichiers entiers. À la fin, tu dois pouvoir expliquer chaque ligne.

LEÇON 27

Mettre en ligne

Quoi	Où (débuter)	Comment
Front React	Vercel ou Netlify	Connecter le dépôt GitHub. Chaque `git push` redéploie automatiquement.
API Express	Railway ou Render	Connecter le dépôt, indiquer la commande de démarrage (`node server.js`) et le port via `process.env.PORT`.
Base de données	PostgreSQL managé (Neon, Supabase, Railway)	En ligne, on remplace SQLite par PostgreSQL. L'adresse de connexion va dans une variable d'environnement.

- Un fichier SQLite peut être effacé à chaque redéploiement sur la plupart des hébergeurs. C'est pour ça qu'en ligne on utilise une vraie base hébergée.
- Le port : `app.listen(process.env.PORT ?? 3001)`.
- Les secrets se configurent dans le tableau de bord de l'hébergeur, jamais dans le code.
- Le front en ligne doit connaître l'adresse de l'API : mets-la dans une variable d'environnement (avec Vite : `VITE_API_URL`). Côté API, autorise ce front avec CORS (paquet `cors`).

Et ensuite ?

Si tu as fini ce cours et le Projet 2 en écrivant le code toi-même, tu sais coder. Tu peux passer au PDF **« Formation Fullstack Pro »** :

- **Module 0** : HTTP, navigateur et TypeScript en profondeur.
- **Module 1** : Next.js, le rendu serveur, la gestion d'état avancée et la performance.
- **Module 2** : les APIs pros, l'authentification et les permissions.

- **Module 3** : PostgreSQL, Prisma, les index et le cache.
- **Module 4** : Docker, la CI/CD, la sécurité et le monitoring.
- **Module 5** : les grosses architectures.

Et à ce moment-là, le vibecoding deviendra une vraie arme : tu sauras *ce que tu demandes* à l'IA et tu pourras *vérifier* ce qu'elle produit.

Checklist de progression

- ☐ Partie 1 : je sais écrire variables, conditions, fonctions, boucles, objets et `map` / `filter` sans aide.
- ☐ Partie 2 : j'ai fait la todo-list seul.
- ☐ Partie 3 : je sais utiliser le terminal, npm, `fetch`, TypeScript et Git.
- ☐ Partie 4 : j'ai refait la todo-list en React, découpée en composants.
- ☐ Partie 5 : j'ai une API qui enregistre dans une base, avec validation et requêtes paramétrées.
- ☐ Partie 6 : mini-lolpanel fonctionne et est en ligne.

Corrections

Il y a souvent plusieurs bonnes solutions. Si la tienne marche et reste lisible, elle est bonne.

Leçon 1

```
console.log("Zelko");
console.log(7 * 6); // 42
```

Leçon 2

```
const pseudo = "Zelko";  // un pseudo change rarement
let niveau = 5;          // augmente
let argent = 120.5;      // varie
let isOnline = true;     // varie
console.log(pseudo, niveau, argent, isOnline);
```

Leçon 3

```
// 1. 17 % 5 = 2  (17 = 3 × 5 + 2)
// 2.
const prix = 30;
const quantite = 3;
console.log(`Total : ${prix * quantite} €`);
// 3. !(5 > 3) → false ; 2 === 2 → true ; false || true → true
```

Leçon 4

```
const note = 14;
if (note < 10) {
  console.log("Raté");
} else if (note <= 13) {
  console.log("Bien");
} else if (note <= 16) {
  console.log("Très bien");
} else {
  console.log("Excellent");
}
```

L'ordre compte : comme on s'arrête au premier cas vrai, on peut tester `<= 13` sans répéter `>= 10`.

Leçon 5

```
const estMajeur = (age) => age >= 18;

function mention(note) {
  if (note < 10) return "Raté";
  if (note <= 13) return "Bien";
  if (note <= 16) return "Très bien";
  return "Excellent";
}

function appliquerReduction(prix, pourcentage) {
  return prix - (prix * pourcentage) / 100;
}
```

Pas besoin de `else` après un `return` : la fonction est déjà terminée.

Leçon 6

```
const jeux = ["Roblox", "Minecraft", "Rocket League", "Fortnite"];
console.log(jeux[1]);      // le 2e = index 1
jeux.push("Zelda");
console.log(jeux.length);  // 5
console.log(jeux[10]);     // undefined : cette case n'existe pas
```

Leçon 7

```
// 1.
for (let i = 10; i >= 1; i--) console.log(i);
console.log("Décollage !");

// 2.
const scores = [12, 45, 7, 30];
let somme = 0;
for (const s of scores) somme += s;
console.log(somme); // 94

// 3.
let max = scores[0];
for (const s of scores) {
  if (s > max) max = s;
}
console.log(max); // 45
```

Leçon 8

```
const bouton = { texte: "Envoyer", couleur: "bleu", largeur: 120,
isVisible: true };

function decrireBouton(b) {
  return `Bouton "${b.texte}" ${b.couleur} de ${b.largeur} px`;
}
console.log(decrireBouton(bouton));
```

Leçon 9

```
const noms = joueurs.map((j) => j.nom);                  //
["Léa","Tom","Sam"]
const forts = joueurs.filter((j) => j.score > 50);       // Léa, Sam
const tom = joueurs.find((j) => j.nom === "Tom");
const total = joueurs.reduce((s, j) => s + j.score, 0);  // 217
```

Leçon 10

1. `Points` avec une majuscule → `ReferenceError`. Il faut `points`.
2. `bonus` est le texte `"5"` : `10 + "5"` donnerait `"105"`. Il faut `const bonus = 5;` ou `Number(bonus)`.
3. `joueur.Score` avec une majuscule → affiche `undefined`. Il faut `joueur.score`.

Leçons 11 à 13

```
<nav class="barre">
  <a class="logo" href="/">Zelko</a>
  <div class="liens">
    <a href="#">Projets</a> <a href="#">À propos</a> <a
href="#">Contact</a>
  </div>
</nav>
```

```
.barre { display: flex; justify-content: space-between; align-items:
center; padding: 12px 16px; }
.liens { display: flex; gap: 16px; }
@media (max-width: 600px) {
  .barre { flex-direction: column; gap: 8px; }
}
```

Leçon 14

```
const affichage = document.querySelector("#valeur");
let n = 0;
function maj() {
  affichage.textContent = n;
  affichage.style.color = n < 0 ? "red" : "";
}
document.querySelector("#plus").addEventListener("click", () => {
n++; maj(); });
document.querySelector("#moins").addEventListener("click", () => {
n--; maj(); });
document.querySelector("#reset").addEventListener("click", () => { n
= 0; maj(); });
maj();
```

Leçon 16

```
const liste = document.querySelector("#liste");
const statut = document.querySelector("#statut");

async function charger() {
  statut.textContent = "Chargement…";
  try {
    const rep = await
fetch("https://jsonplaceholder.typicode.com/users");
    if (!rep.ok) throw new Error(`HTTP ${rep.status}`);
    const users = await rep.json();
    for (const u of users) {
      const li = document.createElement("li");
      li.textContent = u.name;
      liste.append(li);
    }
    statut.textContent = "";
  } catch (e) {
    statut.textContent = "Erreur de chargement. Réessaie.";
  }
}
charger();
```

Leçon 17

```
type Joueur = {
  pseudo: string;
  niveau: number;
  inventaire: string[];
  guilde?: string;
};

function monterNiveau(joueur: Joueur): Joueur {
  return { ...joueur, niveau: joueur.niveau + 1 };
}
```

On renvoie une copie au lieu de modifier l'objet d'origine. C'est exactement ce qu'on fait en React.

Leçon 23

```
app.delete("/api/projets/:id", (req, res) => {
  const id = Number(req.params.id);
  const avant = projets.length;
  projets = projets.filter((p) => p.id !== id);
  if (projets.length === avant) return res.status(404).json({
erreur: "Projet introuvable" });
  res.status(204).end();
});

app.patch("/api/projets/:id", (req, res) => {
  const projet = projets.find((p) => p.id ===
Number(req.params.id));
  if (!projet) return res.status(404).json({ erreur: "Projet
introuvable" });
  const { nom } = req.body;
  if (typeof nom !== "string" || nom.trim() === "" || nom.length >
80) {
    return res.status(422).json({ erreur: "Nom invalide" });
  }
  projet.nom = nom.trim();
  res.json(projet);
});
```

Leçon 24

```
CREATE TABLE joueurs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pseudo TEXT UNIQUE NOT NULL,
  score INTEGER NOT NULL DEFAULT 0
);
INSERT INTO joueurs (pseudo, score) VALUES ('Léa', 80), ('Tom', 45),
('Sam', 92);
SELECT * FROM joueurs WHERE score > 50 ORDER BY score DESC;
UPDATE joueurs SET score = score + 10 WHERE pseudo = 'Tom';
```

Leçon 26

1. **Faux.** N'importe qui peut appeler la route directement. Le serveur doit vérifier que l'utilisateur est admin.

2. **Faux.** Le code du front est téléchargé par chaque visiteur. La clé est lisible par tous, et les robots scannent le web en permanence.
3. **Vrai** pour les valeurs. Mais ça ne remplace pas la vérification des droits : un utilisateur ne doit pas pouvoir lire le projet d'un autre en changeant l'`id`.

> **Rappel :** le but n'est pas d'aller vite, c'est de comprendre chaque ligne. Une leçon bien comprise vaut mieux que trois survolées. Bon code ! 🚀